

Requerimientos Técnicos

Sistema de Ventanilla de Historias Clínicas — Clínica Junical

Versión	Borrador para presentación
Fecha	29 de April de 2026
Audiencia	Dirección administrativa, área de sistemas
Alcance	Hardware, software, integraciones, seguridad y mantenimiento

Este documento describe los recursos técnicos —físicos y lógicos— necesarios para la operación del sistema de ventanilla de historias clínicas, las integraciones con servicios externos, el modelo de seguridad y el plan de mantenimiento. Está redactado de forma resumida y orientada a una toma de decisiones rápida.

1. Requerimientos físicos (hardware)

Servidor (donde corre el sistema)

Componente	Mínimo	Recomendado
Procesador	Intel Core i3 / AMD Ryzen 3	Intel i5 / Ryzen 5
Memoria RAM	4 GB	8 GB
Disco duro	20 GB libres	50 GB SSD
Red	Conexión a internet estable	LAN clínica + internet

Equipos de los usuarios (ventanilla)

Componente	Mínimo
Procesador	Cualquier PC moderno (≥ 2018)
Memoria RAM	4 GB
Pantalla	1366 × 768 mínimo
Navegador	Chrome, Edge o Firefox actual
Periféricos	Teclado, mouse, conexión a red

No requiere instalación de software en los equipos de los usuarios. El acceso es exclusivamente por navegador web.

2. Requerimientos lógicos (software)

Software del servidor

- Sistema operativo: Windows 10/11, Linux (Ubuntu 20.04+) o Windows Server.
- Python 3.11 o superior.
- Node.js 18 o superior (solo para recompilar el frontend).
- PostgreSQL 14 o superior como base de datos.
- Git, para clonado y actualización del proyecto.

Librerías Python principales

- Django 6.x — framework web.
- Django REST Framework — API.
- djangorestframework-simplejwt — autenticación con tokens.
- psycopg2-binary — conector PostgreSQL.
- openpyxl — exportación a Excel.
- reportlab — generación de PDFs (modo simulación de Hosvital).
- requests — llamadas a APIs externas.
- beautifulsoup4 — scraping de fuentes públicas.
- python-dotenv — manejo de variables de entorno.

Librerías Frontend principales

- React 18 + Vite 8.
- React Router DOM 6.
- Axios para consumo de la API.

3. Servicios externos

Servicio	Función	Estado
API Hosvital	Descargar el PDF real de la historia clínica. Pendiente de credenciales.	
Servidor SMTP (Gmail)	Envío de la HC por correo.	Configurable. Funciona con app password.
Twilio WhatsApp	Envío de la HC por WhatsApp.	Opcional. Modo simulado disponible.
Procuraduría / Policía / Contraloría / DANE	Envío de datos del paciente.	Funciona sin credenciales (fuentes públicas).

Cada servicio se configura en el archivo backend/ .env. El sistema funciona aunque alguno de los servicios externos no esté disponible: en ese caso entra en modo de simulación, lo que permite presentar y operar el flujo completo sin bloqueos.

4. Arquitectura general

El sistema sigue una arquitectura cliente-servidor de tres capas: navegador, backend Django y base de datos PostgreSQL. Las integraciones externas se conectan exclusivamente desde el backend.

Capa	Tecnología	Responsabilidad
Frontend	React + Vite	Interfaz de usuario en navegador. Sin instalación local.
Backend	Django + DRF	Lógica de negocio, API REST, autenticación, integraciones.
Base de datos	PostgreSQL	Persistencia de pacientes, solicitudes, usuarios y bitácora.
Almacenamiento	Sistema de archivos (carpetas de red)	PDF reales de las historias clínicas descargados.
Integraciones	Hosvital, SMTP, Twilio, fuentes públicas	Servicios externos consumidos por el backend.

5. Configuración inicial

Comandos del backend

```
cd backend
.\.env\Scripts\Activate.ps1
pip install -r requirements.txt
python manage.py migrate
python manage.py createsuperuser
python manage.py runserver
```

Comandos del frontend

```
cd frontend
npm install
npm run dev
```

Variables del archivo backend/.env

Variable	Descripción
DATABASE_NAME / USER / PASSWORD / HOST / PORT	Credenciales de PostgreSQL.
HOSVITAL_API_URL / HOSVITAL_API_TOKEN	Credenciales de la API de Hosvital.
EMAIL_HOST / EMAIL_HOST_USER / EMAIL_HOST_PASSWORD	Credenciales de SMTP.
TWILIO_ACCOUNT_SID / AUTH_TOKEN / TWILIO_PHONE_NUMBER	Credenciales de Twilio App Business.

6. Seguridad

- Autenticación basada en JWT, con tokens firmados y expiración configurable.
- CORS restringido a las direcciones explícitamente permitidas.
- Bitácora completa de acciones críticas (qué, quién, cuándo).
- Contraseñas almacenadas con hash PBKDF2 (estándar de Django).
- Variables sensibles fuera del código fuente, en archivo .env.
- Conexión recomendada por HTTPS en ambiente productivo.
- Validación de roles y permisos: administrador y ventanilla.

7. Datos almacenados

Entidad	Información principal
Pacientes	Documento, nombre, teléfono, fecha de nacimiento, contacto.
Solicitudes	Los 20 campos del Excel real, estados y PDFs asociados.
Usuarios	Funcionarios con rol (administrador / ventanilla).
Bitácora	Auditoría completa: acción, usuario, fecha, detalle.
PDFs de HC	Archivos descargados desde Hosvital (carpeta media/).

8. Mantenimiento

- Backup diario automatizado de la base de datos PostgreSQL (pg_dump).
- Backup semanal de la carpeta backend/media/ (PDFs guardados).
- Revisión semanal de la bitácora en busca de eventos atípicos.
- Actualización trimestral de dependencias: pip install -U -r requirements.txt.
- Pruebas de restauración del backup al menos una vez al mes.

9. Conclusión

El sistema está diseñado para operar con recursos modestos, sin instalación en los equipos de los usuarios y con tolerancia a la indisponibilidad de servicios externos mediante modos de simulación. Esto permite presentar y comenzar la operación de inmediato, mientras se gestiona la habilitación definitiva de las integraciones con Hosvital, el servidor de correo y WhatsApp Business.